

FinOps & Optimisation Best Practices



Agenda

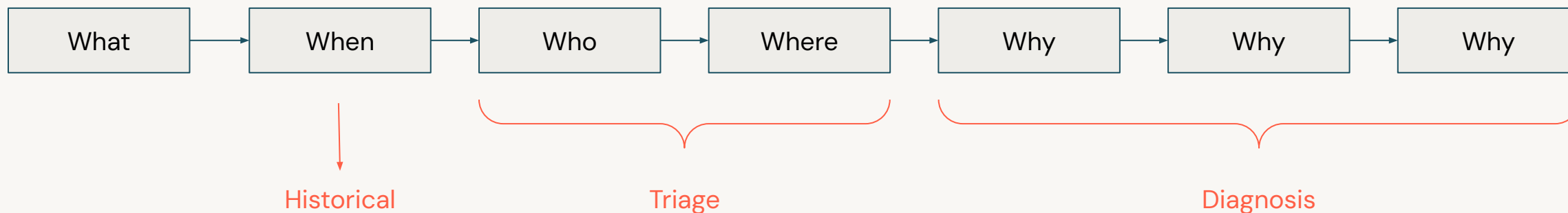
- FinOps & Databricks System Tables
- General Optimization Best Practices
- Clusters and Pools
- Cluster Management & Best Practices
- Cluster Sizing



FinOps & System Tables

Monitoring vs Observability

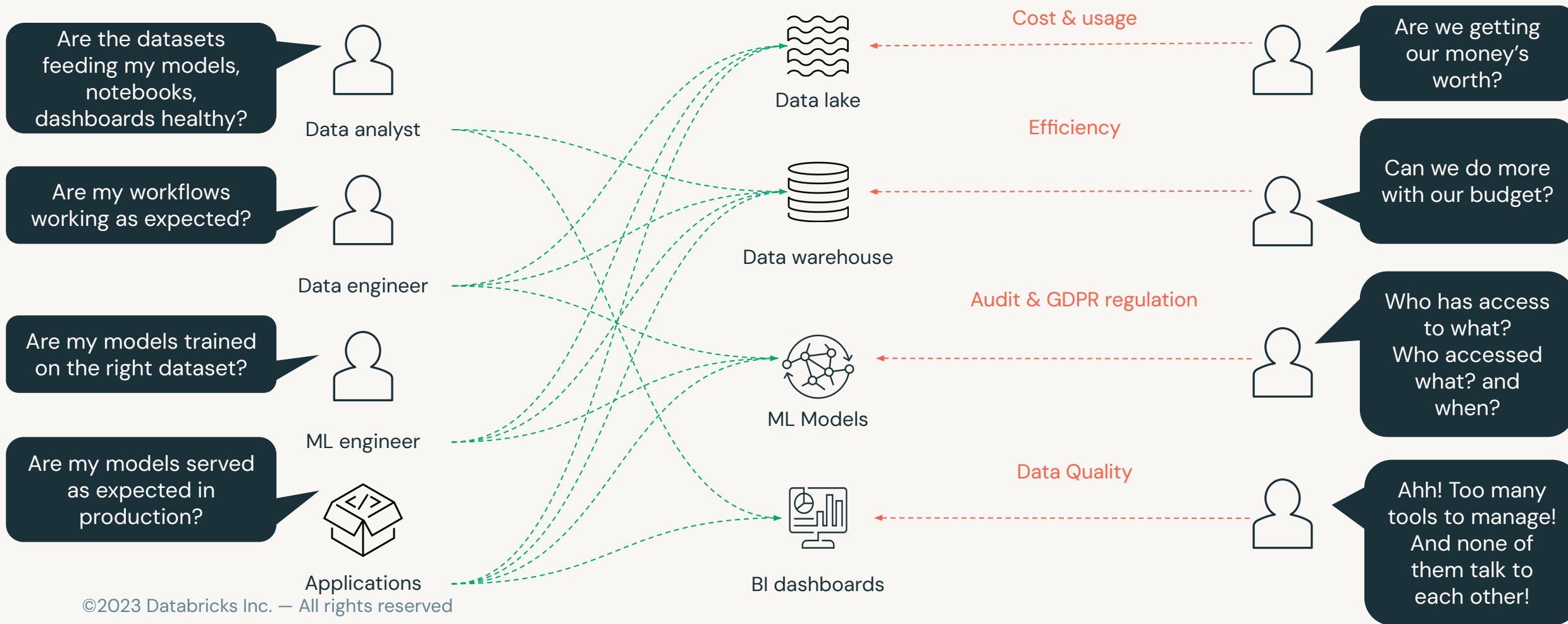
Monitoring	Observability
Tells you whether the system works	Lets you ask why it's not working
Is “the how”/ something you do	Is “the goal”/something you have
An Operational Concern	Embedded at the time of system design
I monitor you	you make yourself observable



Data and AI Observability is complex

Data+AI Consumers/producers

Data+AI Governance Team



...but there's more than one option

Problem statement

Solution

Hot to use

Need for
telemetry &
monitoring

Audit logging

Analyse/Alert using Databricks or send to SIEM

Cluster logging &
metrics

Analyse cluster and spark errors and system metrics such as CPU or memory

System tables

Latest Databricks observability solution for all of Databricks operational data

Overwatch

Advanced solution for cost tracking, anti-pattern detection and optimisation opportunities. Legacy of system tables

Azure Monitor,
Datadog, unravel

3rd party/partners offering which help integrate with existing observability tools

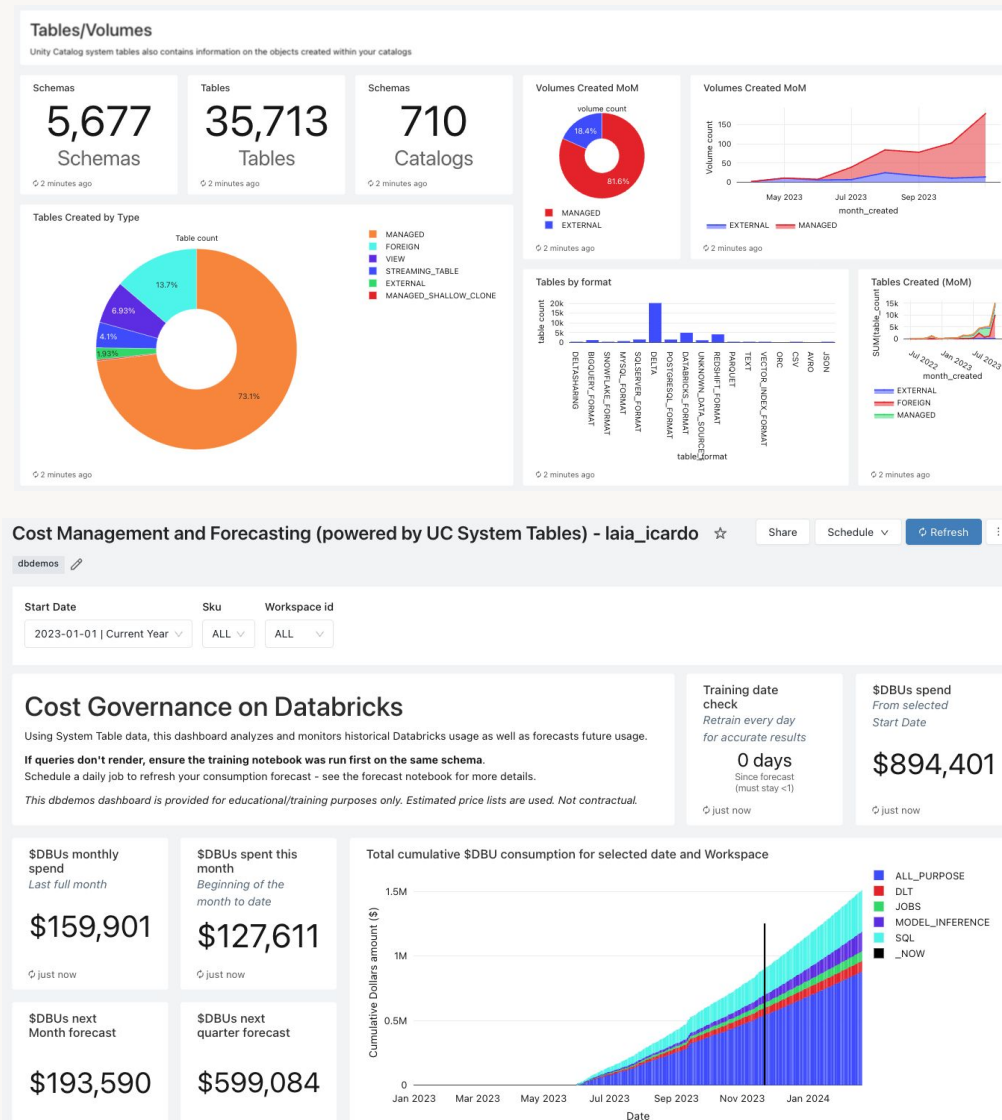


System Tables

Observability with Unity Catalog

Databricks-hosted analytical store for all of Databricks operational data's warm path used for historical customer observability, including:

- **Audit logs:** answer common account usage questions.
- **Billable usage logs:** analyze and optimize consumption.
- **Pricing table:** Pricing information and monitor pricing changes
- **Table and column lineage:** query lineage data
- **Marketplace listing:** operational intelligence for Databricks Marketplace



System Tables – Billing Forecast demo

Observability with Unity Catalog

System Tables: Billing Forecast, Usage Analytics, and Access Auditing With Databricks Unity Catalog

DEMO TYPE

Product Tutorial

DURATION

Self-paced

RELATED CONTENT

[View all tutorials →](#)

[Databricks Unity Catalog: File](#)

To install the demo, get a **free Databricks workspace** and execute the following two commands in a Python notebook

```
%pip install dbdemos
```

```
import dbdemos  
dbdemos.install('uc-04-system-tables')
```

What You'll Learn

Databricks Unity Catalog is the world's first AI-powered governance solution for the lakehouse. It empowers enterprises to seamlessly govern their structured and unstructured data, ML models, notebooks, dashboards, and files on any cloud or platform.

Through Delta Sharing, Databricks Unity Catalog offers direct access to many of the lakehouse activity logs exposed in Delta as System Tables. System Tables are the cornerstone of lakehouse observability and enable at-scale operational intelligence on numerous key business questions. In this demo, we'll show how Unity Catalog System Tables can be used to:

Recommended



TUTORIAL

[Table ACL and Dynamic Views With UC](#)



TUTORIAL

[Access Data on External Locations](#)

https://www.databricks.com/resources/demos/tutorials/governance/system-tables?itm_data=demo_center



Demo: Budget Alerting on System Tables

The screenshot shows the Databricks SQL interface. The query editor contains the following SQL:

```
1 SELECT *
2 FROM system.billing.billing_logs
```

The results table displays the following data:

#	account_id	workspace_id	timestamp	date	cluster_id
1	7a99b43c-b46c-432b-b0a7-814217701909	1061539017586512	2022-06-25 14:59:59.000	2022-06-25	0625-141012-wz7ki0xb
2	7a99b43c-b46c-432b-b0a7-814217701909	1061539017586512	2022-06-28 02:59:59.000	2022-06-28	0628-021018-y9ofklhr
3	7a99b43c-b46c-432b-b0a7-814217701909	1061539017586512	2022-06-28 04:59:59.000	2022-06-28	0628-045018-fvxizg1
4	7a99b43c-b46c-432b-b0a7-814217701909	1061539017586512	2022-06-29 02:59:59.000	2022-06-29	0629-025012-exdiybn
5	7a99b43c-b46c-432b-b0a7-814217701909	1061539017586512	2022-06-27 03:59:59.000	2022-06-27	0627-035022-3yrehhn4
6	7a99b43c-b46c-432b-b0a7-814217701909	1061539017586512	2022-06-30 09:59:59.000	2022-06-30	0630-091010-6vnl9tcd
7	7a99b43c-b46c-432b-b0a7-814217701909	1061539017586512	2022-06-26 09:59:59.000	2022-06-26	0626-091011-qu1aknq6
8	7a99b43c-b46c-432b-b0a7-814217701909	1061539017586512	2022-06-30 01:59:59.000	2022-06-30	0630-012013-qepqpeuv

At the bottom of the results table, it says "4.19 s runtime | 1K rows".

Easily set up alerts on any system table

Understand when you're hitting your budget, when a team or user exceeds a certain amount of spend, or any other query you can dream up!

Supported Destinations:

- Slack
- Teams
- PagerDuty
- Email
- Webhook



Databricks Specialist Sessions

Observability and FinOps with Overwatch and more

Learning Objectives

Databricks Specialist Sessions: Observability and FinOps

You are:

- Someone who actively uses Databricks to build platforms
- Someone with a working knowledge of the current Databricks feature set
- Someone who is a Platform admin/data engineer interested in understanding how to optimize the platform for cost and enable observability.

©2022 Databricks Inc. — All rights reserved

You will get:

- An understanding of observability in the platform using Overwatch system tables.
- A deep-dive into optimisation techniques to reduce cost and improve performance.

Recording

Databricks Specialist Sessions:

Observability and FinOps with Overwatch and more

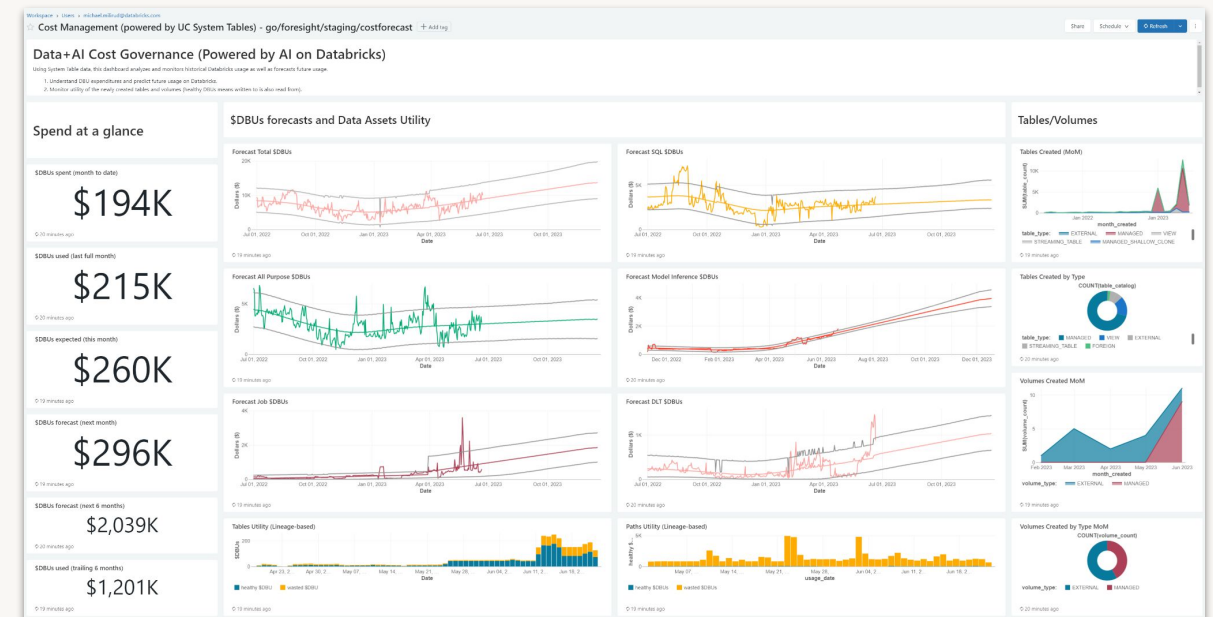
Databricks SSAs
2023-09-27

©2023 Databricks Inc. — All rights reserved

System Tables Roadmap

System Tables are highly evolving to add a wide range of other useful data which makes it a one-stop shop for all observability needs

- Data Marketplace analytics
- Delta Auto Maintenance
- Clusters
- List Prices
- Account / Contract Prices
- Jobs & Tasks
- Warehouse events



Systems table examples



Which tables did a user access?

```
SELECT
  action name as `EVENT`,
  event time as `WHEN`,
  IFNULL(request params.full name arg, 'Non-specific') AS `TABLE` ACCESSED`,
  IFNULL(request params.commandText, 'GET table') AS `QUERY TEXT`
FROM system.access.audit
WHERE user identity.email = {{User}}
      AND action name IN ('createTable', 'commandSubmit', 'getTable', 'deleteTable')
      -- AND datediff(now(), event_date) < 1
      -- ORDER BY event_date DESC
```

Table ▾ + New result table: ON ▾ Q Search □

	A ^B C EVENT	📅 WHEN	A ^B C TABLE ACCESSED	A ^B C QUERY TEXT
1	getTable	2023-12-15T10:56:11.028+00:00	aaron_binns.dopple.messages_raw	GET table
2	getTable	2023-12-15T10:56:11.697+00:00	aaron_binns.dopple.messages_raw	GET table
3	commandSubmit	2023-12-15T10:56:11.598+00:00	Non-specific	> -- this is a system generated query from catalog explorer des
4	commandSubmit	2023-12-15T10:55:17.901+00:00	Non-specific	> DESCRIBE(SELECT * FROM dbdemos.racing_sim.playtime)
5	commandSubmit	2023-12-15T10:55:20.271+00:00	Non-specific	> SELECT timestamp, speed, rpm, throttle, brake, steering, cra
6	commandSubmit	2023-12-15T10:55:20.242+00:00	Non-specific	> DESCRIBE(SELECT timestamp, speed, rpm, throttle, brake, st
7	commandSubmit	2023-12-15T10:58:27.236+00:00	Non-specific	> -- this is a system generated query from catalog explorer des
8	getTable	2023-12-15T10:58:27.325+00:00	aaron_binns.dopple.messages_raw	GET table
9	commandSubmit	2023-12-21T08:05:27.040+00:00	Non-specific	select * from tpcdi.shannon_barrow_tpcdi.dimaccount
10	commandSubmit	2023-12-21T08:05:27.041+00:00	Non-specific	select * from tpcdi.shannon_barrow_tpcdi.dimmessages

How many DBUs of each SKU were used this month?

```
SELECT sku name, usage date,
sum(usage quantity) as `DBUs`
FROM system.billing.usage
WHERE
    month(usage date) = month(NOW())
    AND year(usage date) = year(NOW())
GROUP BY sku_name, usage_date
```

Table ▼ +

New result table: ON ▼

Q

Search

	A B C sku_name	📅 usage_date	1.2	DBUs ↕
1	ENTERPRISE_SERVERLESS_REAL_TIME_INFERENCE_US_WEST_OREGON	2024-01-08		22265.668559782999998899
2	ENTERPRISE_SERVERLESS_REAL_TIME_INFERENCE_US_WEST_OREGON	2024-01-05		21797.486705116555556430
3	ENTERPRISE_SERVERLESS_REAL_TIME_INFERENCE_US_WEST_OREGON	2024-01-04		21094.466734411444444090
4	ENTERPRISE_SERVERLESS_REAL_TIME_INFERENCE_US_WEST_OREGON	2024-01-06		21068.985924708666665720
5	ENTERPRISE_SERVERLESS_REAL_TIME_INFERENCE_US_WEST_OREGON	2024-01-07		21014.746372551444444900
6	ENTERPRISE_SERVERLESS_REAL_TIME_INFERENCE_US_WEST_OREGON	2024-01-03		20627.492271402555555350
7	ENTERPRISE_SERVERLESS_REAL_TIME_INFERENCE_US_WEST_OREGON	2024-01-09		19483.180627598111123875
8	ENTERPRISE_SERVERLESS_REAL_TIME_INFERENCE_US_WEST_OREGON	2024-01-02		19460.050052808666669410
9	ENTERPRISE_SERVERLESS_REAL_TIME_INFERENCE_US_WEST_OREGON	2024-01-01		18815.499308015777776540
10	ENTERPRISE_SERVERLESS_REAL_TIME_INFERENCE_US_WEST_OREGON	2024-01-11		15017.728848458888885231
11	ENTERPRISE_SERVERLESS_REAL_TIME_INFERENCE_US_WEST_OREGON	2024-01-10		11874.826516812888889426
12	ENTERPRISE_SERVERLESS_REAL_TIME_INFERENCE_US_WEST_OREGON	2024-01-12		5297.202217525777787840
13	ENTERPRISE_SERVERLESS_SQL_COMPUTE_US_WEST_OREGON	2024-01-11		4522.204449822222520300
14	ENTERPRISE_SERVERLESS_SQL_COMPUTE_US_WEST_OREGON	2024-01-09		4087.757107522222369971



Which jobs consumed the most DBU?

```
SELECT usage metadata.job_id as `Job ID`, sum(usage_quantity) as `DBUs`
FROM system.billing.usage
WHERE usadata.metadata.job_id IS NOT NULL
GROUP BY `Job ID`
ORDER BY `DBUs` DESC
```

Table ▾ +

	A ^B _C Job ID	1.2 DBUs
1	67237	30816.716041588888188360
2	21237561098793	18562.159699200000616860
3	239329504193850	18414.857476888889498178
4	152487171271780	15145.037225727223554092
5	189806647815195	12902.780245333333790372
6	541188917099106	11891.155924266667072296
7	643380784530544	11473.159191288889316141
8	283528052330009	10795.749487466667026230
9	1114651750039561	10665.517844800000309600
10	499535445443822	7803.202186944444441072

Show me the SKUs growing in usage

```
SELECT after.sku_name, before_dbus, after_dbus, ((after_dbus - before_dbus)/before_dbus * 100) AS growth_rate
FROM
  (SELECT sku name, sum(usage quantity) as before_dbus
    FROM system.billing.usage
   WHERE usage date BETWEEN "2023-04-01" and "2023-04-30"
  GROUP BY sku_name) as before
JOIN
  (SELECT sku name, sum(usage quantity) as after_dbus
    FROM system.billing.usage
   WHERE usage date BETWEEN "2023-05-01" and "2023-05-30"
  GROUP BY sku name) as after
where before.sku name = after.sku_name
SORT by growth_rate DESC
```

Table + New result table: ON Search

	sku_name	before_dbus	after_dbus	growth_rate
1	ENTERPRISE_SERVERLESS_REAL_TIME_INFERENCE_US_EAST_OHIO	260.1880000000000000240	2996.299005046555555730	1051.590000
2	ENTERPRISE_DLT_PRO_COMPUTE	3.435318394444444400	39.484855116666665890	1049.379800
3	ENTERPRISE_JOBS_SERVERLESS_COMPUTE_US_WEST_OREGON	17241.017115688889455085	72274.844541755557966818	319.202900
4	ENTERPRISE_SERVERLESS_SQL_COMPUTE_US_WEST_OREGON	54665.776767544449178364	201729.325387488893513296	269.023100
5	ENTERPRISE_SERVERLESS_REAL_TIME_INFERENCE_EUROPE_IRELAND	5768.04484777777774800	16545.75501111111111900	186.852100
6	ENTERPRISE_SERVERLESS_REAL_TIME_INFERENCE_US_WEST_OREGON	273269.98245444444428158	434592.197318508444459915	59.034000
7	ENTERPRISE_SQL_PRO_COMPUTE_AP_TOKYO	2301.450918666666645150	2726.863674111111058570	18.484500
8	ENTERPRISE_SQL_PRO_COMPUTE_US_WEST_OREGON	2120.851943111111176968	2309.46709466666695500	8.893400
9	> ENTERPRISE_SERVERLESS_REAL_TIME_INFERENCE_LAUNCH_US_WEST_OREG...	1070.000000000000000000	1134.000000000000000000	5.981300
10	ENTERPRISE_SERVERLESS_SQL_COMPUTE_US_EAST_OHIO	417.005836444444458400	425.14091022222237100	1.950800
11	ENTERPRISE_SERVERLESS_SQL_COMPUTE_EUROPE_IRELAND	1235.004055444444352486	1139.69884077777704006	-7.717000
12	ENTERPRISE_DLT_CORE_COMPUTE	358.163794022222217965	329.69479761666663754	-7.948600
13	ENTERPRISE_DLT_ADVANCED_COMPUTE_(PHOTON)	18089.772245601666570907	15501.706715616666473363	-14.306800
14	ENTERPRISE_ALL_PURPOSE_COMPUTE_(PHOTON)	60899.569780566667034720	51178.429108061111017214	-15.962600

What consumption is assigned to resources with a certain tag?

```
SELECT sku name, usage unit, SUM(usage_quantity) as `DBUs consumed`
FROM system.billing.usage
WHERE custom tags.{{key}} = "{{value}}"
GROUP BY 1, 2
```

Databricks

Search data, notebooks, recent, and more...

+
UI preview
Send feedback

dbxbench-sab

ConfigurationNotebooks (0)LibrariesEvent logSpark UIDriver logsMetricsAppsSpark compute UI - Master

Access mode

No isolation shared

Performance

Databricks Runtime Version

14.1 (includes Apache Spark 3.5.0, Scala 2.12)

☐ Use Photon Acceleration

Node type

m6gd.large8 GB Memory, 2 Cores

New Use fleet instance types for improved spot placement and availability Learn more

☐ Enable autoscaling local storage

☐ Terminate after 0 minutes of inactivity

Instance profile

shard-demo-s3-access

Tags

NoAutoRemoveTrue

AllowDowntimeoff

KeepAliveTrue

Table +

	A ^B C sku_name
1	ENTERPRISE_ALL_PURPOSE

Table			
	A ^B _C sku_name	A ^B _C usage_unit	1.2 DBUs consumed
1	ENTERPRISE_ALL_PURPOSE_COMPUTE	DBU	4998.9443045305555648760
2	ENTERPRISE_ALL_PURPOSE_COMPUTE_(PHOTON)	DBU	25.544224916666667000

Which cluster owners use the most DBUs?

```
SELECT
  u.record id record id,
  c.cluster id cluster id,
  max by(c.owned by, c.change time) owned_by,
  max(c.change time) change time,
  any value(u.usage start time) usage start time,
  any_value(u.usage_quantity) usage_quantity
FROM
  system.billing.usage u
  JOIN system.compute.clusters c
WHERE
  u.usage metadata.cluster id is not null
  and u.usage start time >= '2023-01-01'
  and u.usage metadata.cluster id = c.cluster_id
  and c.change_time <= u.usage_start_time
GROUP BY 1, 2
ORDER BY cluster_id, usage_start_time desc;
```

Table +

New result table: ON

Search

	A ^B record_id	A ^B cluster_id	A ^B owned_by	🕒 change_time	🕒 usage_start_time	1.2	usage_quantity
1	9f26547a-1565-3707-a7be-10c3fba2120e	0101-000045-w4iol2bh	mwc@databricks.com	2024-01-01T00:00:45.809+00:00	2024-01-01T03:00:00.000+00:00		1.441254308888889000
2	20aaf2c1-b792-3ce8-b5bf-10a524f0f42a	0101-000045-w4iol2bh	mwc@databricks.com	2024-01-01T00:00:45.809+00:00	2024-01-01T02:00:00.000+00:00		7.9460000000000001000
3	6d5c3ade-5138-3365-a5c5-7d5760fa366a	0101-000045-w4iol2bh	mwc@databricks.com	2024-01-01T00:00:45.809+00:00	2024-01-01T01:00:00.000+00:00		7.9460000000000001000
4	e36c0d52-fd46-3787-9d38-b2f89746cf3c	0101-040851-poqiguym	brian.law@databricks.com	2024-01-01T04:08:51.258+00:00	2024-01-01T14:00:00.000+00:00		1.9497498666666666000
5	fdac82d9-d492-300c-914f-d65b2d231e6d	0101-040851-poqiguym	brian.law@databricks.com	2024-01-01T04:08:51.258+00:00	2024-01-01T13:00:00.000+00:00		22.0800000000000000000
6	de858ef7-7838-37ea-b21d-78f17ec93371	0101-040851-poqiguym	brian.law@databricks.com	2024-01-01T04:08:51.258+00:00	2024-01-01T12:00:00.000+00:00		22.0800000000000000000
7	31aee226-40b4-35ed-9249-427fe0e934b1	0101-040851-poqiguym	brian.law@databricks.com	2024-01-01T04:08:51.258+00:00	2024-01-01T11:00:00.000+00:00		7.5021277333333325000
8	acaa2600-6f3c-3561-911b-8b83b13d4d4e	0101-040851-poqiguym	brian.law@databricks.com	2024-01-01T04:08:51.258+00:00	2024-01-01T06:00:00.000+00:00		21.6662269333333300000
9	f398461f-db1c-3586-a04b-c65e706a0cca	0101-040851-poqiguym	brian.law@databricks.com	2024-01-01T04:08:51.258+00:00	2024-01-01T05:00:00.000+00:00		22.0800000000000000000
10	5d23d668-2f38-32b9-b369-8cc71ac139b1	0101-073032-9k9ffhp9	mwc@databricks.com	2024-01-01T07:30:33.018+00:00	2024-01-01T08:00:00.000+00:00		1.1370375888888890000
11	d667edd1-ae0e-3ca0-ad7c-0f0bfc6481a1	0101-080045-cesj2t5b	mwc@databricks.com	2024-01-01T08:00:45.149+00:00	2024-01-01T11:00:00.000+00:00		6.9177213833333340000

```
select usage_quantity as dbu_used_ratio
from system.billing.usage
```

```
select sum(usage_quantity) / 400000 as dbu_used_ratio
from system.billing.usage
```

```
select sum(usage_quantity) / 400000 as dbu_used_ratio
from system.billing.usage
where usage date > date trunc("month", now())
```

Exemple 1 : Notification

As an administrator, I want to receive a notification if the monthly budget is about to be consumed

	.00	dbu_used_ratio
1		0.836809240445895006

databricks

Search data, notebooks, recents, and more...

+ P

Workspace > Users > artem.sheiko@databricks.com

Edit alert

Alert name

Max budget used by month

Query

Billing usage monthly

Trigger condition

Value column

dbu_used_ratio

Operator

>

Threshold value

0.8

When query result has no rows, set state to UNKNOWN

[Preview alert](#)

Notifications

When alert is triggered

Send notification Just once

When alert returns back to normal

☐ Send notification

Template

Use default template

TRIGGERED This alert would trigger with the current data

dbu_used_ratio	Threshold
0.836809240445895	0.8



Example 2: Identify long-running queries and track inefficient code running through Notebooks with System Tables and Lakeview

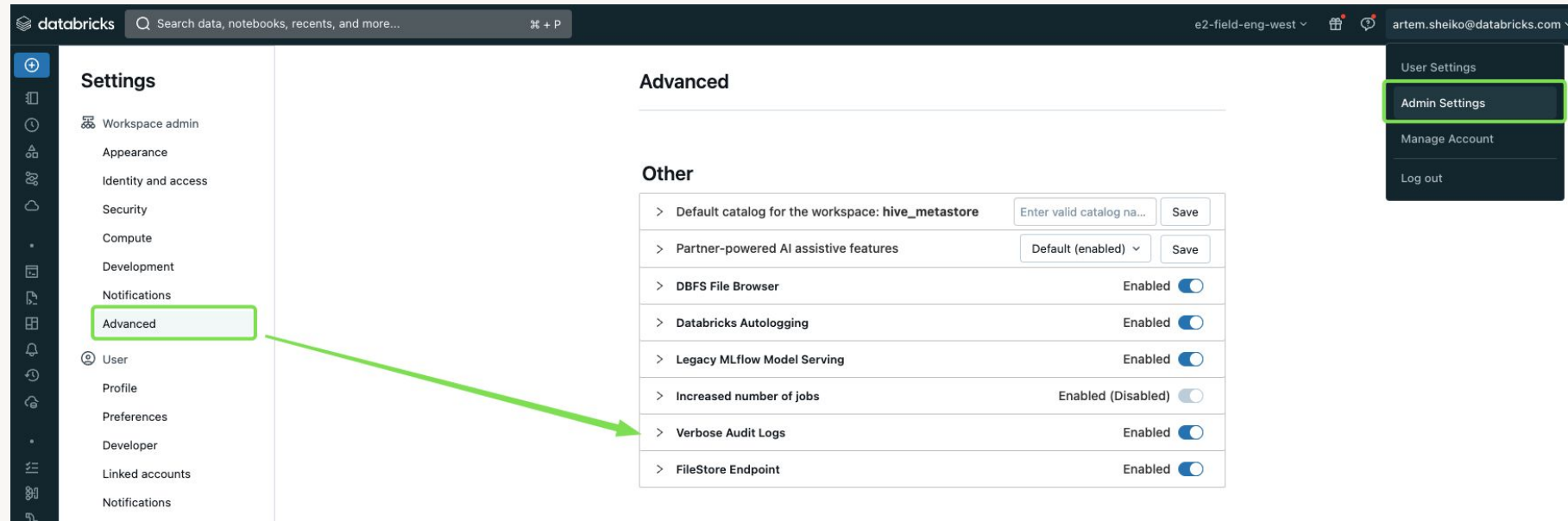
The events we need are tracked with the commandSubmit, commandFinish, runCommand events (see Databrickssql logs reference for more information).

databrickssql	commandSubmit	Only in verbose audit logs. Runs when a command is submitted to Databricks SQL.	- warehouseId - commandId - validation - commandText
databrickssql	commandFinish	Only in verbose audit logs. Runs when a command completes or a command is cancelled.	- warehouseId - commandId
notebook	runCommand	Available when verbose audit logs are enabled. Emitted after Databricks runs a command in a notebook. A command corresponds to a cell in a notebook. executionTime is measured in seconds.	- notebookId - executionTime - status - commandId - commandText - commandLanguage



Example 2: Identify long-running queries and track inefficient code running through Notebooks with System Tables and Lakeview

Step 1 : Activate verbose audit logs



The screenshot shows the Databricks Admin Settings interface. On the left, the 'Settings' sidebar has 'Advanced' highlighted under the 'Notifications' section. A green arrow points from this 'Advanced' link to the 'Advanced' settings panel. The 'Advanced' panel is titled 'Advanced' and contains a section 'Other' with several settings. The 'Verbose Audit Logs' setting is highlighted with a green box and is currently 'Enabled' with a blue toggle switch.

Other	
> Default catalog for the workspace: hive_metastore	Enter valid catalog na... Save
> Partner-powered AI assistive features	Default (enabled) Save
> DBFS File Browser	Enabled ☒
> Databricks Autologging	Enabled ☒
> Legacy MLflow Model Serving	Enabled ☒
> Increased number of jobs	Enabled (Disabled) ☐
> Verbose Audit Logs	Enabled ☒
> FileStore Endpoint	Enabled ☒

Once enabled, Databricks will populate the `system.access.audit` table with the events in question. We can then run the queries and create visualizations to spot areas where we need to pay more attention to reduce costs.



Example 2: Identify long-running queries and track inefficient code running through Notebooks with System Tables and Lakeview

Step 2 : Prepare dataset

```
SELECT
  event_date,
  user_identity.email,
  request_params.notebookId, request_params.clusterId, request_params.executionTime,
  request_params.status, request_params.commandLanguage, request_params.commandId,
  request_params.commandText
FROM system.access.audit
WHERE 1=1
  AND action_name = 'runCommand'
  AND request_params.status NOT IN ('skipped')
  AND TIMESTAMPDIFF(HOUR, event_date, CURRENT_TIMESTAMP()) < 24 * 90
ORDER BY request_params.executionTime DESC
```

```
SELECT
  action_name as `EVENT`,
  event_time as `WHEN`,
  IFNULL(request_params.full_name_arg, 'Non-specific') AS `TABLE ACCESSED`,
  IFNULL(request_params.commandText, 'GET table') AS `QUERY TEXT`
FROM system.access.audit
WHERE user_identity.email like '<>@databricks.com'
  AND action_name IN ('createTable', 'commandSubmit', 'getTable', 'deleteTable')
  AND datediff(now(), event_date) < 1
```

Raw results ▾ +

New result table: ON ▾

	📅 event_date	👤 email	📖 notebookId	📖 clusterId	🕒 executionTime	🏁 status	🗨️ commandLanguage	📄 commandId	📄 commandText
1	2023-12-05	firas.farah@databricks.com	1148550101330641	1205-194753-y00mvvj8	999.984	finished	python	1148550101330680	> # A metric refresh will a
2	2024-01-02	mark.oswald@databricks.com	undefined	0102-060103-zgggu5vg	999.804	finished	python	8498768322581739946	> import sys sys.argv = ['
3	2023-12-31	mark.oswald@databricks.com	undefined	1231-150104-2itxdspr	999.751	finished	python	4877666688404071588	> import sys sys.argv = ['
4	2023-10-24	masataka.yokota@databricks.com	3498831615282896	5162b1f21be8a8d8	999.75	finished	sql	79297106391693	> MERGE INTO field_den

Raw results ▾ +

New result table: ON ▾

	🏁 EVENT	📅 WHEN	📖 TABLE ACCESSED	📄 QUERY TEXT
104	commandSubmit	2024-01-16T10:57:28.527	Non-specific	> -- This is a system generated query from sql editor select * from `system`.`information_s...
105	commandSubmit	2024-01-16T10:58:12.192	Non-specific	> WITH q AS (SELECT event_date, user_identity.email, request_params.notebookId, request...
106	commandSubmit	2024-01-16T10:58:12.176	Non-specific	> WITH q AS (SELECT event_date, user_identity.email, request_params.notebookId, request...
107	getTable	2024-01-16T10:57:26.872	system.access.audit	GET table
108	getTable	2024-01-16T10:49:13.098	system.billing.usage	GET table
109	getTable	2024-01-16T10:49:12.292	system.billing.usage	GET table

Example 2: Identify long-running queries and track inefficient code running through Notebooks with System Tables and Lakeview

Step 3 : Create dashboard

New

Workspace

Recents

Catalog

Workflows

Compute

SQL

SQL Editor

Queries

Dashboards

Databricks

Search data, notebooks, recents, and more...

+ P

e2-field-eng-west

artem.sheiko@databricks.com

Dashboards

Dashboards

Lakeview Dashboards New

Filter dashboards

Last modified

Owner: Me (artem.sheiko@databricks.com)

Name	Owner	Last modified
Long running notebook commands	Artem Sheiko	Jan 16, 2024, 08:32 AM
Long running queries / commands	Artem Sheiko	Jan 12, 2024, 12:15 PM
New Dashboard 2023-12-15 11:54:59	Artem Sheiko	Dec 15, 2023, 11:55 AM
New Dashboard 2023-12-07 13:21:56	Artem Sheiko	Dec 07, 2023, 01:21 PM

View samples gallery

Create Lakeview dashboard

Databricks

Search data, notebooks, recents, and more...

+ P

e2-field-eng-west

artem.sheiko@databricks.com

Canvas

Data

Send feedback

New Dashboard ☆ (Saved)

Serverless Sh... Serverless M

Refresh

Share

⋮

Datasets

Notebook commands exec time

Create another dataset

Select a table

Create from SQL

rename the query

Run

```
1 SELECT
2   event_date,
3   user_identity.email,
4   request_params.notebookId, request_params.clusterId, request_params.executionTime,
5   request_params.status, request_params.commandLanguage, request_params.commandId,
6   request_params.commandText
7 FROM system.access.audit
8 WHERE 1=1
9   AND action_name = 'runCommand'
10  AND request_params.status NOT IN ('skipped')
11  AND TIMESTAMPDIFF(HOUR, event_date, CURRENT_TIMESTAMP()) < 24 * 90
12 ORDER BY request_params.executionTime DESC
13
```

Click on “Run” to execute the query at least once

Example 2 : Identify long-running queries and track inefficient code running through Notebooks with System Tables and Lakeview

Step 3 : Create dashboard

New

Workspace

Recents

Catalog

Workflows

Compute

SQL

SQL Editor

Queries

Dashboards

Alerts

Query History

SQL Warehouses

Data Engineering

Job Runs

Data Ingestion

Delta Live Tables

Machine Learning

Playground

Experiments

Features

Models

Serving

Previews

Marketplace

Canvas

Data

Send feedback

New Dashboard

☆

(Saved)

Serverless Sh...

Serverless

M

Refresh

Share

⌵

⋮

Select at least one field to create a visualization.

⏏

↶

📄

🔍

Dataset

Notebook commands exec time

Visualization

Table

Title

Description

Columns

Grid

Display row number

event_date

email

notebookId

clusterId

executionTime

status

commandLanguage

commandId

commandText

©2024 Databricks Inc. — All rights reserved

Example 2 : Identify long-running queries and track inefficient code running through Notebooks with System Tables and Lakeview

Step 3 : Create dashboard

The screenshot shows the Databricks Lakeview interface. On the left is a sidebar with navigation options like Workspace, Recents, Catalog, Workflows, Compute, SQL, SQL Editor, Queries, Dashboards, Alerts, Query History, SQL Warehouses, Data Engineering, Job Runs, Data Ingestion, Delta Live Tables, Machine Learning, Playground, Experiments, Features, Models, Serving, Previews, Marketplace, and Partner Connect. The main area is titled 'Canvas' and shows a table of query execution data. The table has columns: event_date, email, notebookid, clusterid, executionTime, status, commandLanguage, and com. The 'notebookid' column is highlighted with a green box. A green arrow points from the 'notebookid' column to the 'notebookid' field in the right-hand configuration panel. The configuration panel shows 'notebookid' selected as the field to display, with a 'Link' display type and a URL template 'https://e2-demo-field-eng.clou'.

event_date	email	notebookid	clusterid	executionTime	status	commandLanguage	com
2023-12-05	firas.farah@databricks.com	1148550101330641	1205-194753-y00mvvj8	999.984	finished	python	1148
2024-01-02	mark.oswald@databricks.com	undefined	0102-060103-zgggu5vg	999.804	finished	python	8498
2023-12-31	mark.oswald@databricks.com	undefined	1231-150104-2itxdspr	999.751	finished	python	4875
2023-10-24	masataka.yokota@databricks.com	3498831615282896	5162b1f21be8a8d8	999.75	finished	sql	7925
2023-11-02	srijit.nair@databricks.com	1728814240326294	1101-133434-91b0dj5k	999.427	failed	python	2488
2024-01-15	peyman.nasirifard@databricks.com	1312111372358663	0105-124123-ec64hiv0	9984.143	cancelled	python	1312
2023-10-31	sriharsha.jana@databricks.com	1728814240149638	1006-044241-86c08xzw	9982.092	cancelled	python	1728
2023-11-02	shikhar.datta@databricks.com	undefined	1101-133434-91b0dj5k	999.427	failed	python	2488

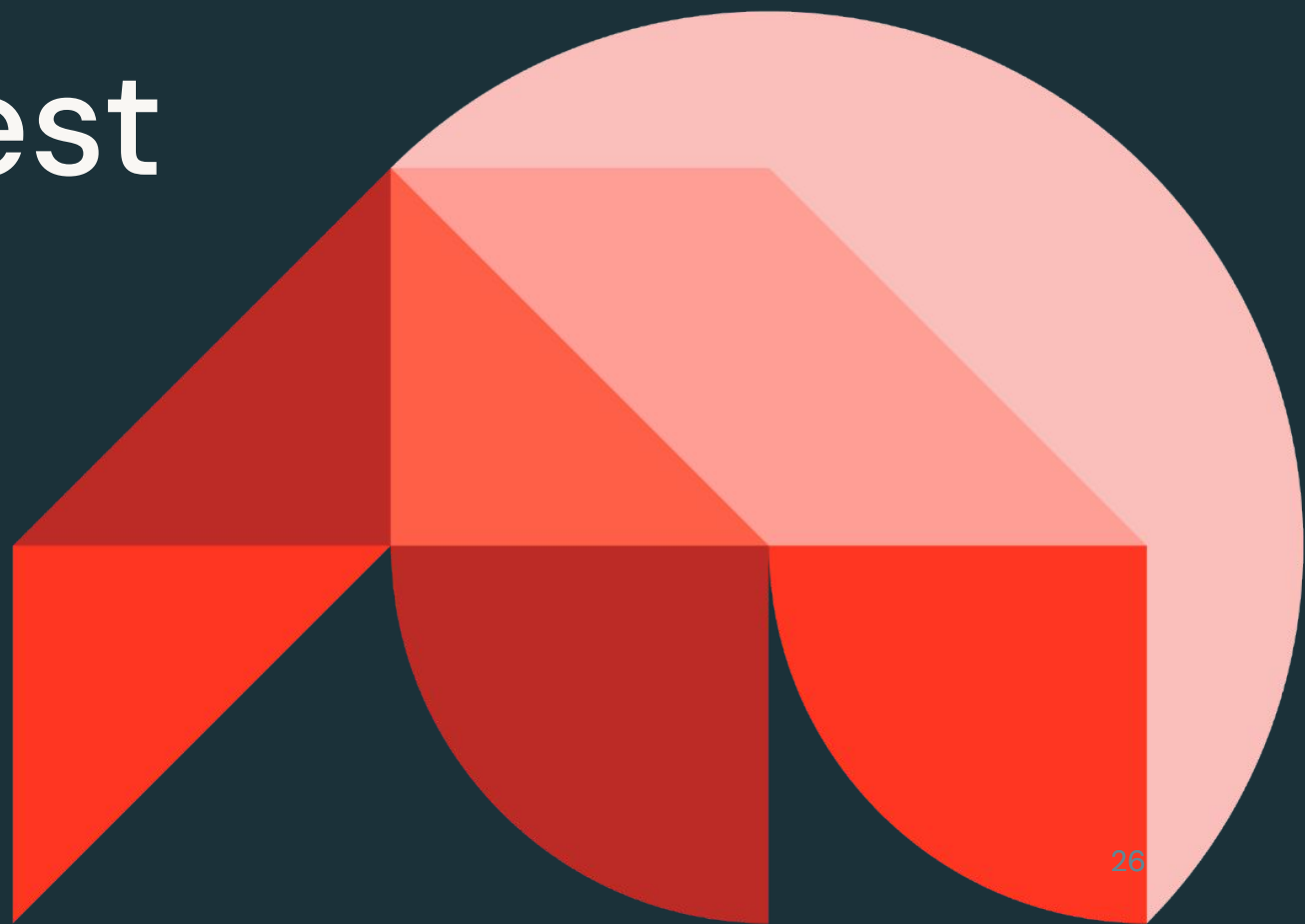
In order to be able to transform the contents of the columns to the links allowing you to open the Notebook / cluster in question, URL templates for the respective columns must be composed as:

notebookId: `https://<workspace name>.azuredatabricks.net/?o=<workspace id>#notebook/{{ @ }}`

clusterId: `https://<workspace name>.azuredatabricks.net/?o=<workspace id>#setting/clusters/{{ @ }}`

Optimization Best Practices

April 2024



General recommendations

Mindset when processing data



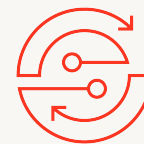
I/O – Use Delta Lake

- Read the least amount of data to get the correct answer
- Interact with your storage system as efficiently as possible



Network – Choose effective joins, use AQE

- Move data as minimally as possible
- Join the smallest tables first, then the next largest, then broadcasts



CPU – Use Photon and size your cluster correctly

- Do the most efficient set of operations to get the same answer
- Avoid inefficient or expensive operations (don't spill)
- Getting things to run in Photon completely almost always solves the problem

Clusters Top Tuning Tips

Performance Optimization (1)

- Use the **latest LTS** version of Databricks Runtime
 - The latest Databricks runtime is almost always faster than the one before it
 - DBR 10.0 Upgraded Hadoop libs to 3.3, Photon operator coverage increased
 - DBR 9.1 Enhanced merge algorithm for 2–3x faster merges (Public Preview)
 - DBR 8.3 Photon Public Preview
 - DBR 7.6 upgraded zorder mechanism, auto tune merge
 - DBR 7.3 significantly reduced delta metadata overhead time
 - DBR 7.2 added cloning, upgraded cloud connectors, upgraded AQE
- Use Photon – fastest Spark execution engine written in C++ ([launch blog](#))
 - DBU cost is 50–100% higher but if performance is >2x then the number of nodes or cores/node can be reduced saving Compute cost while having the same or better performance
- Restart clusters periodically, at least once per week (or even per day on busy clusters)
 - Some older DBRs might have stuck / zombie Spark jobs
- Check the cluster Event log
 - Check for Spot failures, Driver GCs, lost nodes, etc. It's probably the first thing you should do

Clusters Top Tuning Tips

Performance Optimization (2)

- Use the **Delta Cache Memory optimized instances** instead of normal Memory instances
 - They even have similar or less cost while having significant CPU and storage improvements
 - For example, use [Standard_E16ds_v4](#) instead of [Standard_DS14_v2](#) (unless you must use reserved instances):
 - Standard_E16ds_v4 (\$1.152/hour) is cheaper than Standard_DS14_v2 (\$1.196/hour) and same DBU cost
 - Standard_E16ds_v4 has better configuration (16 Cores, 128GB Memory) than Standard_DS14_v2 (16 cores, 112GB Memory)
 - Standard_E16ds_v4 has bigger and faster local SSD with Delta Cache while Standard_DS14_v2 has none of that
- But also try [Ddsv4](#)-series (General purpose) for less intensive memory jobs:
 - Standard_D16ds_v4 (\$0.904/hour) – same cores and SSD as Standard_E16ds_v4 but half memory for lower price
 - Photon not yet supported on these instances, so best option for non-Photon workloads
 - The SSD is fast enough on these instances, Delta Cache can be enabled manually:
 - `spark.databricks.io.cache.enabled true`
 - `spark.databricks.io.cache.maxDiskUsage 322122547200`
 - `spark.databricks.io.cache.compression.enabled true`
- Rules of thumb
 - Keep total memory available to the machine less than 64GB (or 128GB on memory intensive or Photon workloads)
 - For ad-hoc queries / BI workloads – aim for Standard_E8ds_v4 / Standard_E16ds_v4 Delta Cache instances
 - For ETL – use Standard_D8ds_v4 / Standard_D16ds_v4 unless having large spills or benefits from Photon

Clusters Top Tuning Tips

Cost Optimization

- Don't use Interactive Clusters for Jobs
 - Use ephemeral Job Clusters for Jobs
 - This is the single biggest cost optimization impact: the DBU cost is 50% for a Job Cluster compared with an Interactive one
- Use Cluster Autoscaling
 - Use minimum of 1 to reduce cost at the expense of user experience
 - Use Instance Pools to improve user experience
 - Increase spark.databricks.aggressiveWindowDownS to improve user experience
- Use Cluster Automatic termination to terminate a cluster after a specified inactivity period in minutes for cost saving
 - Use Instance Pools to improve user experience
- Use Spot VMs to use spare Azure VM instances for below market rate
 - Great for ad-hoc/shared clusters but not recommended for production jobs
- Configure a large driver node (4–8 cores and 16–32GB is mostly enough) *only* if:
 - Have large datasets being returned / collected (spark.driver.maxResultSize is typically increased also)
 - Large broadcast joins
 - Many (>50) streams or concurrent jobs / notebooks on the same cluster

Delta Top Tuning Tips

Performance & Cost Optimization

- Use [Delta](#) instead of Parquet or text formats
- Use [Delta Cache](#)
 - Transparent Cache of Parquet data on fast local SSDs – no need to manage the cache or JVM memory
 - Best to use Accelerated instances with Delta Cache enabled by default (such as [Standard E16ds v4](#))
- Use [Liquid Clustering](#)
- Alternatively (if Liquid Clustering not used):
 - Don't over-partition (>100 partitions)
 - Don't have partitions with <1GB of data (in each partition)
 - Use [Auto Optimize](#) (both Optimized Writes and Auto Compaction)
 - [Z-Order](#) your tables by up to 3–5 columns
 - First by common filter columns, second by join keys
- Make sure you use [Data skipping](#)
 - By default, only the 32 columns are used for data skipping (can be increased at an overhead for writes)
- Use [Predictive Optimization](#) to intelligently optimize the table data layouts
- Alternatively (if Predictive Optimization not used), run periodic maintenance jobs:
 - [Manual compaction](#): `OPTIMIZE table_name`
 - [Vacuum](#): `VACUUM table_name`
 - [Collect statistics](#): `ANALYZE TABLE table_name COMPUTE STATISTICS FOR ALL COLUMNS`

Spark Top Tuning Tips

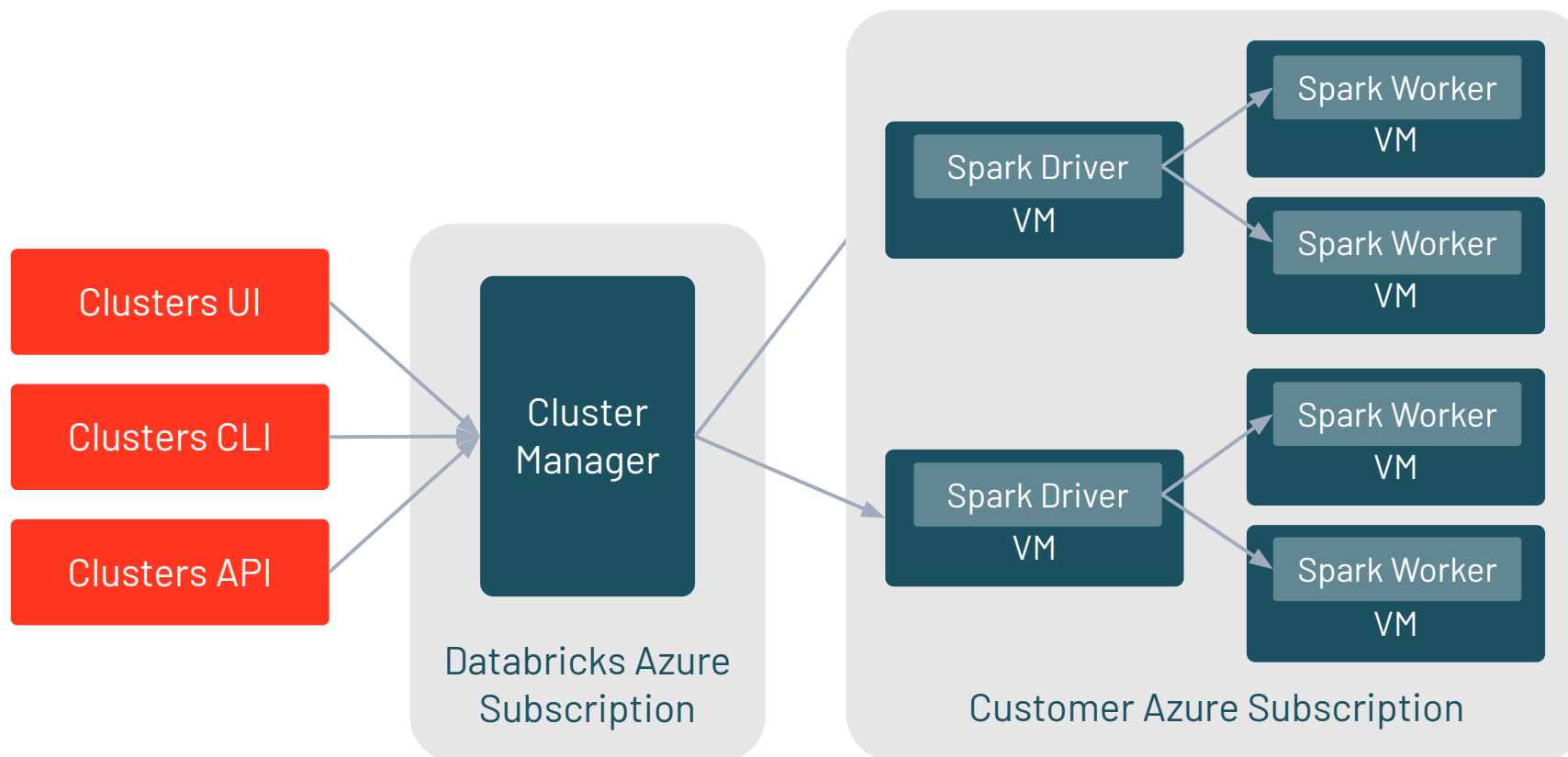
Performance & Cost Optimization

- Start by finding out the best candidate code for optimizations
 - Usually these are notebooks or code called multiple times from various pipelines (different being a job parameter)
 - And pipelines or notebooks that takes a majority of the job processing time
- Use Spark 3.x (DBR >7.3) to benefit from [Adaptive query execution](#)
- Set the shuffle partitions to a multiple of number of cores
 - Default is 200 which is wasteful for clusters with > 200 cores:
 - `spark.conf.set("spark.sql.shuffle.partitions", 2 * sc.defaultParallelism)`
 - If Autoscaling, use the maximum number of nodes multiplied by the number of cores per node
 - Try the new [Auto Optimized Shuffle](#): `spark.conf.set("spark.sql.shuffle.partitions", "auto")`
- Optimize joins
 - Increase `spark.sql.autoBroadcastJoinThreshold` to `100*1024*1024`
 - Set `spark.sql.join.preferSortMergeJoin` to `false`
 - Use [join hints](#)
 - Thoroughly test the above in case it causes memory issues in edge cases
- Don't use [Spark Caching](#) unless absolutely necessary – use the automated [Delta Caching](#)
 - Spark Caching is normally useful when caching reads from JDBC or at the end of complex computations
- Remove unnecessary `count`, `display`, `show`, `collect`

Clusters and Pools

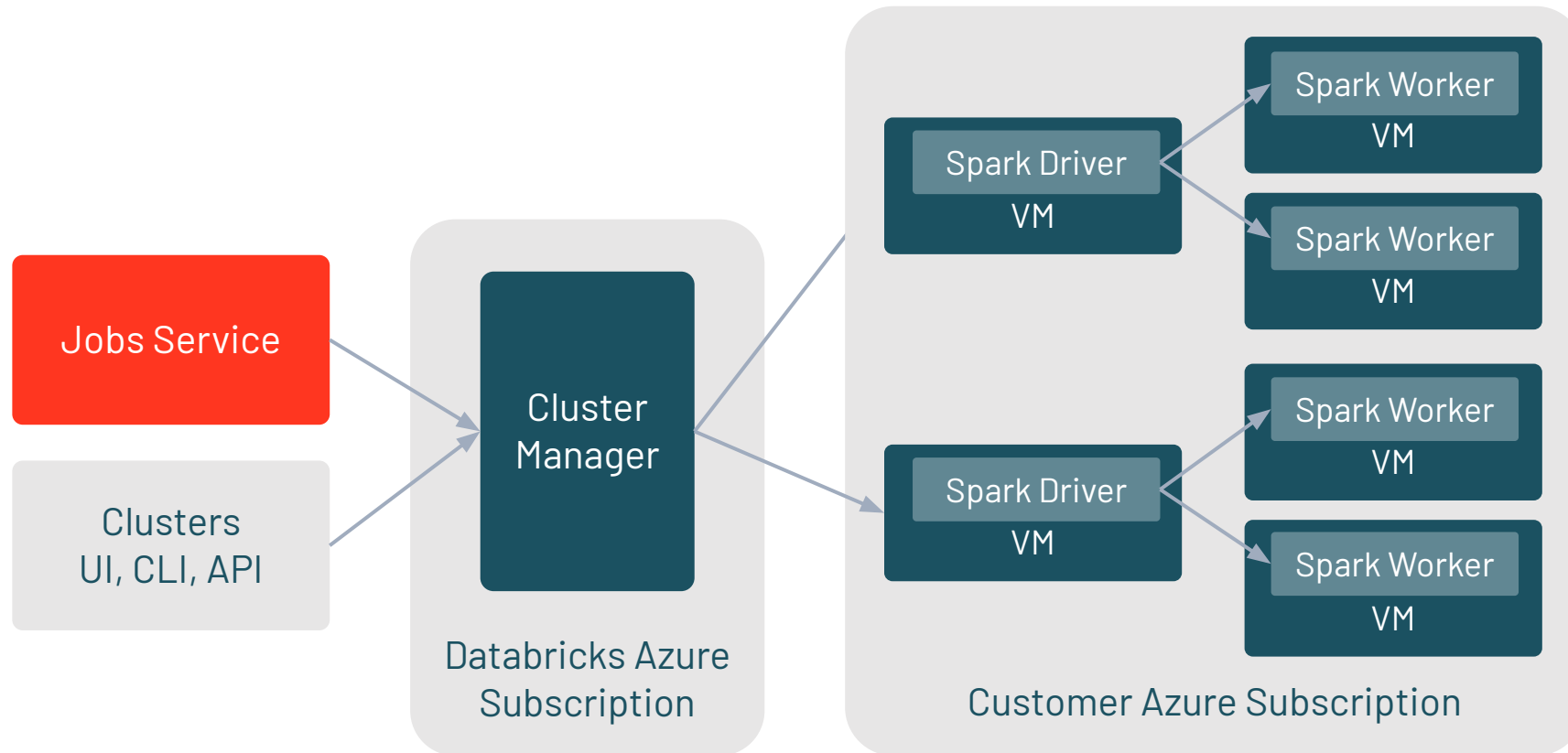
Cluster

Managed with the UI, CLI, or REST API



Job

A non-interactive way of running a notebook/library immediately or on a scheduled basis



Using Databricks Efficiently

ALL PURPOSE COMPUTE

- Analysis and ad-hoc work
- Shared clusters but best practice is to separate by team or workload
- Anytime an already-running cluster is utilized (including API or scheduled)
- More expensive

JOBS COMPUTE

- Run on ephemeral clusters that are created for the job, and terminate on completion
- Pre-scheduled or submitted via API
- Single-user
- Great for isolation and debugging
- Production and repeat workloads
- Lower cost

Cluster Types

Type	All-Purpose Cluster	Jobs Cluster
Persistence	Persistent cluster	Ephemeral cluster created for job, terminated on completion
Workload	Interactive (data analytics)	Automated (data engineering)
Use	Analysis and ad-hoc work	Production and repeated workloads
Benefits	Collaborate interactive analysis, Ability to restart cluster	Isolation and debugging, Scheduled runs
Cost	\$\$\$	\$

Cluster Modes – Comparison

	Standard	High Concurrency	Single Node
Users	Single user recommended	Multi-user (separate process per user)	Single user recommended
Languages	Python, R, Scala, SQL	SQL, Python, R	Python, R, Scala, SQL
Python and SQL Table Access Control	No	Yes	No
Credential Passthrough	Limited to single user	Only allow Python and SQL commands	Limited to single user
Process Isolation	No	Yes	No

Cluster Modes – Use Cases

	Standard	High Concurrency	Single Node
Use Cases	Production batch or streaming ETL/ML jobs, Ad-hoc development	Analytics or BI, Ad-hoc development	Lightweight EDA, Single node ML workloads
Audience	Data engineer or data scientist	Data analysts, management, finance	Data Analyst or Data Scientist

Guardrails for Concurrency

Query Watchdog

- Process prevents queries from monopolizing cluster resources
- Examines most common causes of large queries and terminates queries that pass a threshold
- Enabled for all all-purpose clusters created using the UI

Task Preemption

- Scheduler automatically preempts tasks to enforce fair sharing of resources
- Guarantees interactive response times on clusters with many concurrently running jobs

Pools

Why

- Save instance acquisition time; many libraries and dependencies can make this significant
- Prevents short jobs from wasting more time starting than running

Costs

- Azure Databricks does not charge DBUs for idle instances
- Cloud provider costs still apply
- Limits can be configured to manage costs

Cost Control with Pools

Configure your pool to meet your needs and minimize costs

Scenario	Recommendation
Interactive workloads run during business hours only	Set Min Idle count to zero after hours
Automated data pipeline runs for a few hours at night	Option 1: Set Min Idle count a few minutes before the pipeline starts, then revert it to zero afterwards Option 2: Always keep a Min Idle of zero, but set Idle Instance Auto Termination timeout to meet your needs
Optionally, budget VM resources by setting Maximum Capacity for the pool to limit the sum of all idle instances and instances used by clusters attached to the pool	

Spot VMs

- Use spot instances to use spare VM instances for below market rate
 - Great for ad-hoc/shared clusters
 - Not recommended for production with tight SLAs
 - Never use for driver!
- Combine on-demand and spot instances (with custom spot price) to tailor clusters to different use cases

SLA	Spot or On-Demand
Non-mission critical jobs	Driver on-demand and workers spot
Workflows with tight SLAs	Use spot instance w/fallback to on-demand
Production jobs	Use on-demand instances

Cluster Management and Best Practices

Autoscaling

- Dynamically resizes cluster based on workload
 - Can run faster than a statically-sized, under-provisioned cluster
 - Can reduce overall costs compared to a statically-sized cluster

Clusters /

Cancel Confirm DBU / hour: 6 - 18 ?

Policy ?
Unrestricted | v

Cluster name

Cluster mode ?
Standard | v

Databricks runtime version ? [Learn more](#)
Runtime: 9.1 LTS (Scala 2.12, Spark 3.1.2) | v

☐ Use your own Docker container ?

Autopilot options

☒ Enable autoscaling ?

☐ Enable autoscaling local storage ?

☒ Terminate after minutes of inactivity ?

Use Case	Autoscaling Range
Ad-hoc usage or business analytics	Large variance
Production batch jobs	Not needed or buffer on upper limit
Streaming	Don't use yet!

Automatic Termination

- Terminates a cluster after a specify inactivity period in minutes for cost savings
- Enable auto termination for ad-hoc clusters to prevent idle clusters from running overnight or on weekends
- Standard clusters default to auto terminate in 120 minutes
- High concurrency clusters DO NOT auto terminate by default

Clusters /

Cancel Confirm DBU / hour: 6 - 18 ?

Policy ?

Unrestricted | v

Cluster name

Cluster mode ?

Standard v

Databricks runtime version ? [Learn more](#)

Runtime: 9.1 LTS (Scala 2.12, Spark 3.1.2) | v

☐ Use your own Docker container ?

Autopilot options

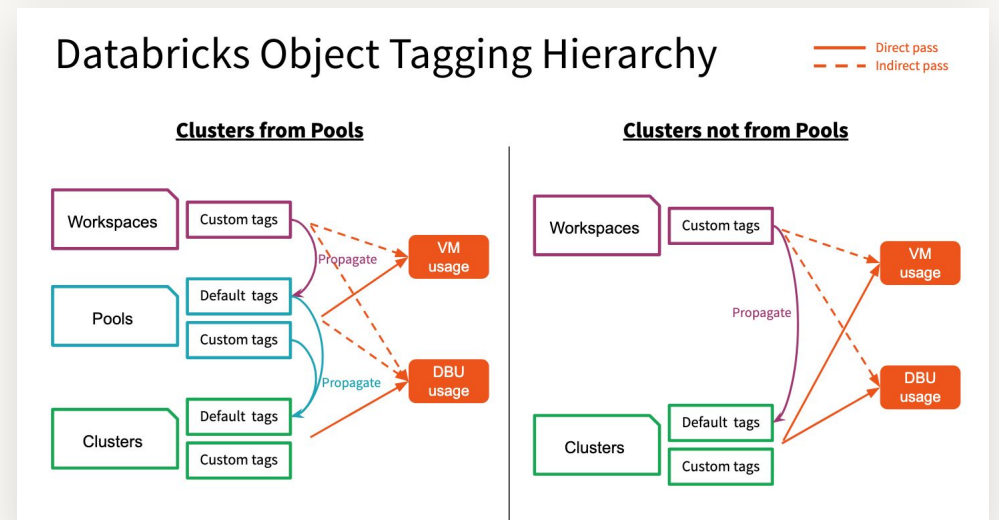
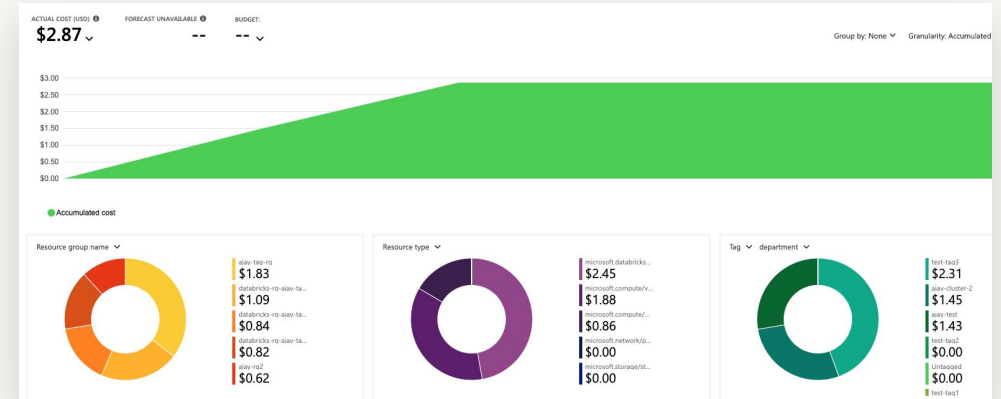
☒ Enable autoscaling ?

☐ Enable autoscaling local storage ?

☒ Terminate after 120 minutes of inactivity ?

Department Chargeback with Cluster Tags

- Cluster tags can be associated with Databricks clusters to attribute costs to specific teams/departments
- Cluster tags propagate to the Azure Cost Report to easily visualize these costs by department in reports or to download or access via API or Overwatch for further analysis.
- Cluster tags also propagate to VMs to be able to attribute department DBU spend with department VMs spend.



Manage Cluster Permissions with ACLs

- Cluster Access Control allows you to control who can create Databricks clusters.
- Granular cluster-level permissions give you control over whether users or groups can manage, access, and/or have restart permissions over clusters.
- Combined, these capabilities allow you to limit cluster creation to a set of admins who can manage and govern access to clusters.

Permission Settings for: **New Cluster**

Who has access:

admins (group)	Can Manage	
Alice (alice@mycompany.com)	✓ No Permissions	✕
	Can Attach To	
	Can Restart	✕
	Can Manage	

Add Users and Groups:

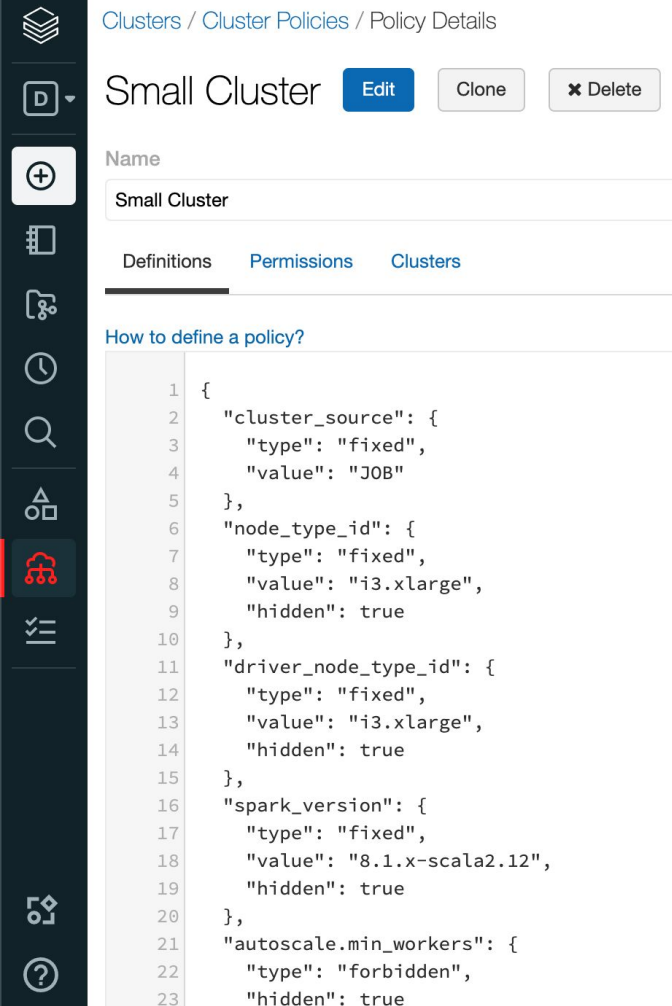
Can Attach To

Username	Name	Admin	Workspace access	Databricks SQL access	Allow unrestricted cluster creation ?
alice@example.com	Alice Doe	☐	☒	☒	☐
bob@example.com	Bob Roe	☐	☒	☒	☐



Predictable Costs with Cluster Policies

- Cluster Policies allow you to mandate users or groups to follow pre-defined rules when configuring clusters.
- Policies can limit the size and type of cluster as well as set max DBU per hour limit on clusters.
- Can automatically enforce tags for department chargebacks on cluster creation.
- By leveraging shared clusters, cluster policies, and tags, interactive clusters can be created for each department with a max DBU/hour limit, and charged back to that department for **consistent cost management**.



The screenshot displays the Databricks 'Cluster Policies / Policy Details' page for a policy named 'Small Cluster'. The interface includes a sidebar with navigation icons and a main content area. The 'Small Cluster' policy is shown with buttons for 'Edit', 'Clone', and 'Delete'. Below the name, there are tabs for 'Definitions', 'Permissions', and 'Clusters'. A section titled 'How to define a policy?' shows a JSON configuration for the policy.

```
1 {
2   "cluster_source": {
3     "type": "fixed",
4     "value": "JOB"
5   },
6   "node_type_id": {
7     "type": "fixed",
8     "value": "i3.xlarge",
9     "hidden": true
10  },
11  "driver_node_type_id": {
12    "type": "fixed",
13    "value": "i3.xlarge",
14    "hidden": true
15  },
16  "spark_version": {
17    "type": "fixed",
18    "value": "8.1.x-scala2.12",
19    "hidden": true
20  },
21  "autoscale.min_workers": {
22    "type": "forbidden",
23    "hidden": true
24  }
25 }
```



Cluster Governance Best Practices

Restrict

- Restrict cluster creation rights
- Reduce users with Can Manage permission
- Use Can Restart or Can Attach To instead
- Use Cluster Policies to enforce rules during cluster creation

Apply Policies

- Enforce specific cluster sizes for different workload or analysis needs
- Enforce specific runtime standard across users/business units (LTS)
- Provide certified power users access to mostly unrestricted policy
- Create different policies for jobs and interactive clusters

Cluster Sizing

Choose the instance type

For existing workloads

- Start by choosing a newer instance type while matching the same cluster size and node size – don't want to disturb a history of previous sizing
 - Standard_E16ds_v4 (16 Cores, 128GB Memory) instead of Standard_DS14_v2 (16 cores, 112GB Memory)
 - Standard_E8ds_v4 (8 Cores, 64GB Memory) instead of Standard_DS13_v2 (8 Cores, 56GB Memory)
 - Standard_D16ds_v4 (16 Cores, 64GB Memory) instead of Standard_DS5_v2 (16 Cores, 56GB Memory) or Standard_D16s_v3 (16 Cores, 64GB Memory)
- Then monitor and use a cheaper instance type depending on the workload
 - For example, use a newer General purpose type such as Standard_D16ds_v4 (16 Cores, 64GB Memory) instead of Standard_E16ds_v4 (16 Cores, 128GB Memory) – this will be cheaper for smaller cost (unless really benefiting from Photon or having workloads with large disk spills)
- Check the workload type
 - For ad-hoc queries / BI workloads – aim for Standard_E8ds_v4 / Standard_E16ds_v4 Delta Cache instances
 - For ETL – use Standard_D8ds_v4 / Standard_D16ds_v4 unless having large spills or benefits from Photon

Choose the instance type

For new workloads and jobs

- Just a series of IFTTT (If This Then That) questions and rules of thumb!
 - Side note – if you 2x the cluster and it runs in 1/2 the time, it costs the same
- Rules of thumb
 - First run: set `spark.sql.shuffle.partitions` = 2x # of cores
 - Keep total memory available to the machine less than 64GB (or 128GB on memory intensive or Photon workloads)
 - Avoid setting any other configs at first (don't carry over configs from legacy platforms unless necessary)
 - Number of cores should be a ratio of 1 core to 128MB → 2GB of reads
 - Some caveats may apply

Scan parquet +details	
Stages: 655.0	
file sorting by size time	2 ms
cache writes size (uncompressed) total (min, med, max)	262.4 MiB (576.8 KiB, 1031.5 KiB, 1085.7 KiB)
time spent in the cache locality manager in milliseconds total (min, med, max)	37 ms (0 ms, 0 ms, 37 ms)
number of files read	1,027
filesystem read data size total (min, med, max)	279.8 MiB (620.5 KiB, 1101.1 KiB, 1155.4 KiB)
cache async file status fetch waiting time total (min, med, max)	0 ms (0 ms, 0 ms, 0 ms)
scan time total (min, med, max)	14.6 m (1.9 s, 3.2 s, 7.1 s)
filesystem read data size (sampled) total (min, med, max)	279.8 MiB (620.5 KiB, 1101.1 KiB, 1155.4 KiB)
filesystem read time (sampled) total (min, med, max)	12.4 m (1.7 s, 2.8 s, 5.5 s)
metadata time	8 ms
size of files read	236.7 GiB

Selecting Driver Node instance type

- Drivers often don't use that many resources – they don't do intensive tasks like the workers since they typically act solely as a coordinator for the worker nodes
- This is true except, but not limited to, cases where:
 - Large datasets are being returned to the driver (`spark.driver.maxResultSize` was increased)
 - The job is broadcasting large variables or doing broadcast joins – these variables have to be collected to the driver first and then sent to all the workers thus sufficient memory size on the driver is crucial
 - Parallel/Threading tasks are being executed on the driver through parallelized arrays or threading
 - Many (hundreds) of queries are being executed at the same time – CPU is important here
 - Using the cluster for non-Spark tasks such as Pandas and other Python libraries
- The five points above are some of the examples where larger drivers are helpful
- The key point here is that generally speaking, the drivers don't need to be very large but just keep in mind the work that is to be performed by the driver and size it accordingly

Selecting Worker Node instance type

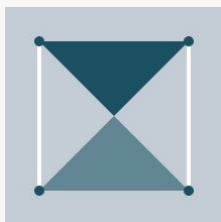
What you should care about with the instance type

- Core to Ram ratio
- Processor type
- Local vs remote storage and storage medium (SSDs vs HDDs)

Workload Type	Azure Type	Core:Ram	CPU	Storage	Recommended Use Case
Memory Optimized (normal)	DSv2	1:7GB	Intel Ice / Cascade Lake	Slow SSD	Memory-intensive applications
Memory Optimized (Delta Cache)	Edsv4	1:8GB	Intel Ice / Cascade Lake	Fast SSD	Memory-intensive applications and Databricks SQL / BI workloads
Compute Optimized	Fsv2	1:2GB	Intel Ice / Cascade Lake	Slow SSD	Structured Streaming, Distributed Analytics, Data Science Applications
Storage Optimized	Lsv2	1:8GB	AMD EPYC	Fastest NVMe SSD	Use cases that require higher disk throughput and IO (large joins)
General Purpose	Ddsv4	1:4GB	Intel Ice / Cascade Lake	Fast SSD	Enterprise-grade applications, relational databases, and analytics with in-memory caching

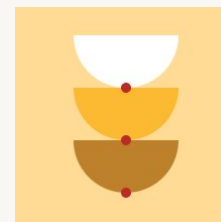
IFTTT – Step 1

Want to use Photon



No

Next slide



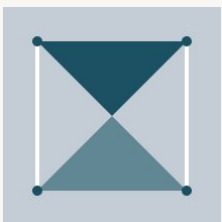
Yes

Edsv4:

- Standard_E4ds_v4
- Standard_E8ds_v4
- Standard_E16ds_v4

IFTTT – Step 2

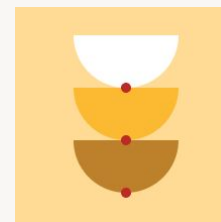
Is your job an ETL job that uses joins/windows/groupbys/aggregations



No

Fs or Fsv2:

- Standard_F8s
- Standard_F16s
- Standard_F8s_v2
- Standard_F16s_v2



Yes

Fs or Fsv2:

- Standard_F8s
- Standard_F16s
- Standard_F8s_v2
- Standard_F16s_v2

IFTTT – Step 3

Run the job with the instance type, follow our rules of thumb and go to the SQL UI of the longest running query – do you see spill?

HashAggregate +details	
spill size	0.0 B
time in aggregation build total (min, med, max)	15.7 m (2.1 s, 3.5 s, 7.4 s)
peak memory total (min, med, max)	64.3 MiB (256.0 KiB, 256.0 KiB, 256.0 KiB)
passthrough output rows	0
avg hash probe bucket list iters	0
rows output	514

No:

- Stop, this is good enough

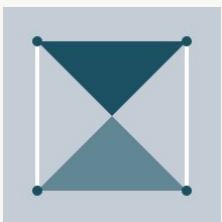
Tasks: Succeeded/Total	Input	Output	Shuffle Read ▾	Shuffle Write
1/1			156.0 KiB	
1/1			136.8 KiB	
1/1			132.1 KiB	
1/1			131.9 KiB	

Yes:

- Set spark.sql.shuffle.partitions to the largest shuffle read stage / 200MBytes
- Or spark.sql.shuffle.partitions=auto

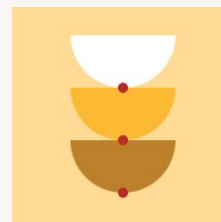
IFTTT – Step 4

Run the job with the updated shuffle partitions – do you still see spill?



No

- Stop, this is good enough



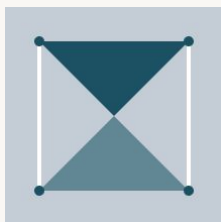
Yes

Ddsv4:

- Standard_D4ds_v4
- Standard_D8ds_v4
- Standard_D16ds_v4

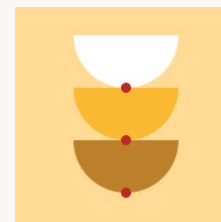
IFTTT – Step 5

Run the job with the updated instance type – do you still see spill?



No

- Stop, this is good enough



Yes

Edsv4:

- Standard_E4ds_v4
- Standard_E8ds_v4
- Standard_E16ds_v4

How many nodes?

- Generally, enable auto-scaling 1 → n nodes
 - Apart from streaming – streaming does not support downscaling
- Fewer big instances > more small instances; reduces network shuffle
 - Around 8-16 is a rule of thumb (better to scale up first but not more than 64GB per node or 128GB for memory intensive jobs)
 - Applies to batch ETL mainly (for streaming, one could start with smaller instances depending on complexity of transformation)
 - Not set in stone, and reverse would make sense in many cases – so sizing exercise matters
- Allow more partitions to be solved simultaneously by increasing node count
- No point having more available CPUs than partitions to process
- Watch for data skew (<10% of partitions requiring >80% of job time) – more nodes will not help unless a repartition is added to the DAG

Data Skew – Identification

- Below is a screenshot of some well distributed data. Highly skewed data would see a difference between the 25th and 75th percentiles of $\sim > 20\%$

Summary Metrics for 10990 Completed Tasks

Metric	Min	25th percentile	Median	75th percentile	Max
Duration	34 s	1.7 min	2.4 min	3.0 min	7.5 min
GC Time	0 ms	0.5 s	0.8 s	2 s	20 s
Input Size / Records	0.0 B / 910450	121.7 MB / 1018717	122.3 MB / 1083023	122.9 MB / 1094265	123.2 MB / 1161127

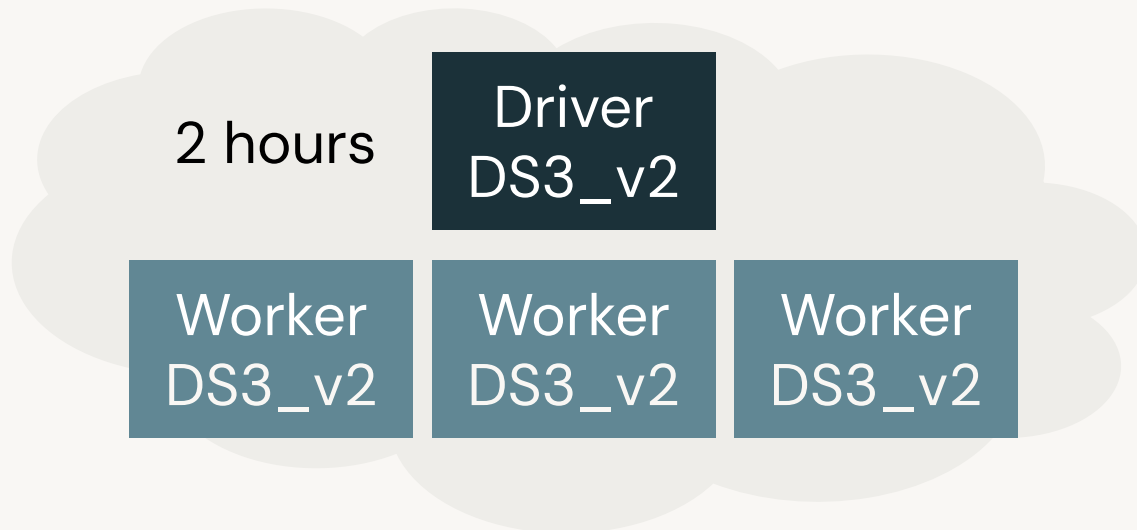
- $(122.9 - 121.7) / 122.9 = < 1\%$ difference
- Note that skew on smaller datasets is less of an issue, the problem grows exponentially with data size

Azure Databricks Cost Management

Azure Databricks Costs – DBUs

A Databricks Unit (DBU) is a unit of processing capability per hour, billed on per-second usage

- Charged upon launch of a Databricks cluster
- No DBUs charged for idle pool instances (Azure VM costs still apply)



$4 \text{ nodes} \times 2 \text{ hours} = 8 \text{ DBUs}$

Total cost = 8 DBU cost + Azure cost of 8 instance hours of DS3_v2



Resource Groups per Workspace



> Delete

^ Essentials

Status : Deploying

Resource group : [demogroup](#)

Location : North Central US

Subscription : [DB Training](#)

Subscription ID : ebd6f6ff-3d3c-4bb9-8ad1-489868582b7f

Managed Resource Group : [databricks-rg-demoworkspace-ghxrhcixtw6oa](#)

Pricing Tier : standard

User Identified Resource Group
(where workspace is deployed)

Managed Resource Group (where all
infrastructure resources are created)



Tags

The screenshot displays the Databricks workspace interface. On the left, the 'demoworkspace' sidebar shows various details: Status: Deploying, Resource group: demogroup, Location: North Central US, Subscription: DB Training, Subscription ID: ebd6f6ff-3d3c-4bb9-8ad1-489868582b7f, and Tags: department : education. A blue callout box points to the 'Tags' field, stating: 'Workspace Tags – applied upon creation of workspace, inherited by resources associated w/workspace'. On the right, the 'demo-cluster' configuration page is shown with tabs for Configuration, Notebooks, Libraries, Event Log, and Spark UI. The 'Tags' tab is active, displaying a table of tags. A blue callout box points to the 'department' tag in the table, stating: 'Cluster Tags – applied upon creation of cluster, inherited by resources associated w/cluster'. A red double arrow points from the 'Tags' field in the workspace sidebar to the 'Tags' tab in the cluster configuration page.

Workspace Tags – applied upon creation of workspace, inherited by resources associated w/workspace

Cluster Tags – applied upon creation of cluster, inherited by resources associated w/cluster

Workspace Details:

- Status : Deploying
- Resource group : demogroup
- Location : North Central US
- Subscription : DB Training
- Subscription ID : ebd6f6ff-3d3c-4bb9-8ad1-489868582b7f
- Tags (change) : department : education

Cluster Configuration (demo-cluster):

- Managed Resource Group : databr
- Pricing Tier : standa

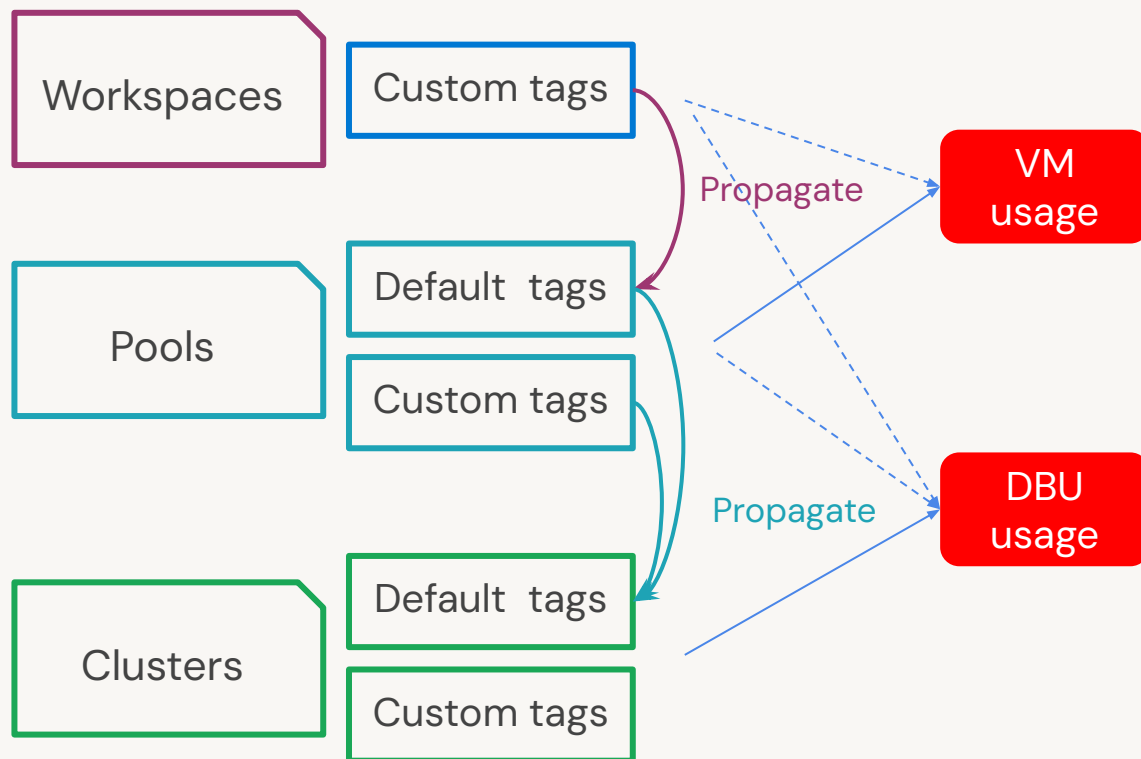
Tags Table:

Key	Value
ClusterId	1206-233634-hypo566
ClusterName	demo-cluster
Creator	
Vendor	Databricks
department	education

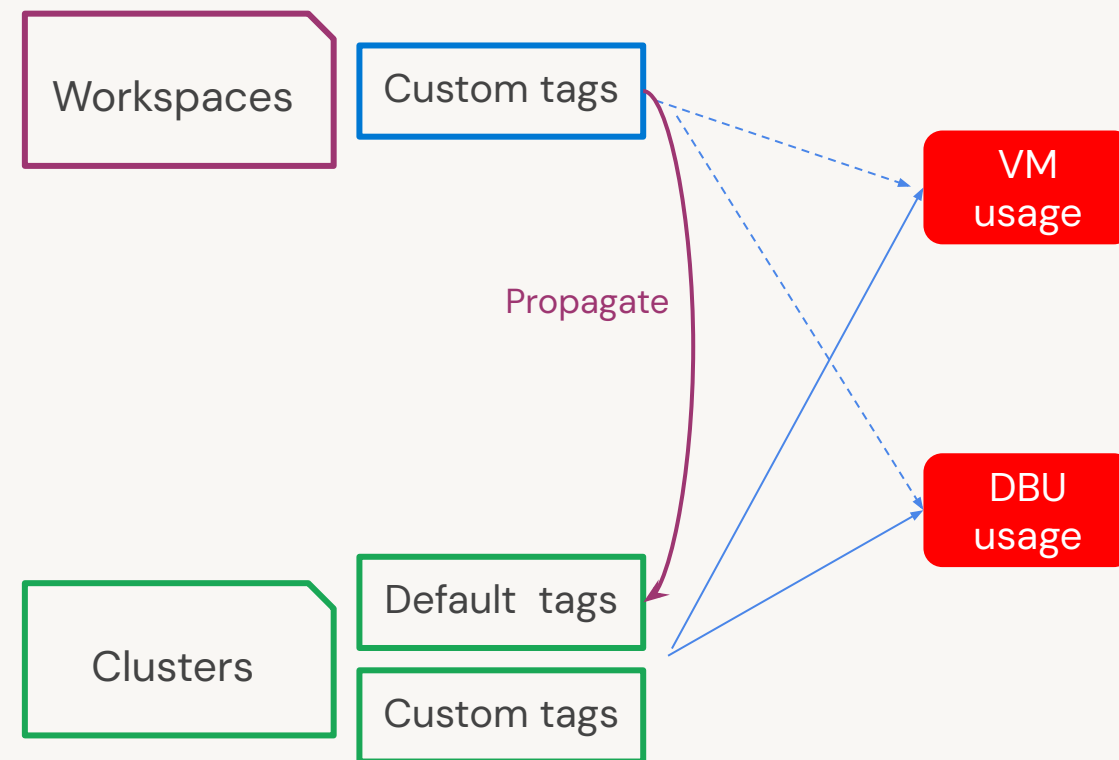
Databricks Object Tagging Hierarchy

Clusters from Pools do not propagate cluster tags to VMs!

Clusters from Pools



Clusters not from Pools



Cost Report

ACTUAL COST (USD) ⓘ

\$2.87 ✓

FORECAST UNAVAILABLE ⓘ

--

BUDGET:

-- ✓

Group by: None ▾ Granularity: Accumulated



● Accumulated cost

Resource group name ▾



ajay-taq-rq	\$1.83
databricks-rq-ajav-ta...	\$1.09
databricks-rq-ajav-ta...	\$0.84
databricks-rq-ajav-ta...	\$0.82
ajav-rq2	\$0.62

Resource type ▾



microsoft.databricks...	\$2.45
microsoft.compute/v...	\$1.88
microsoft.compute/...	\$0.86
microsoft.network/p...	\$0.00
microsoft.storage/st...	\$0.00

Tag ▾ department ▾



test-taq3	\$2.31
ajav-cluster-2	\$1.45
ajav-test	\$1.43
test-taq2	\$0.00
Untagged	\$0.00
test-taq1	\$0.00

Cost Export

Customers also ask how to get breakdowns by DBUs at a SKU level, to build custom reporting solutions on top of the data.

- Via [CSV Cost Exports](#)
- Via native [Billing APIs](#) (only available to Enterprise Agreement customers)

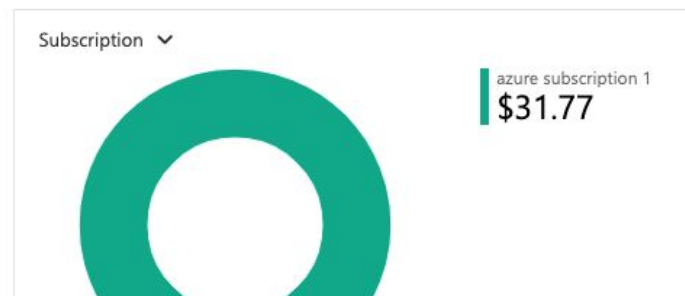


Pre Purchase Plan (P3)

- Allows organizations to pre-purchase DBUs with additional benefits:
 - Volume discounts, service credits, training credits, Slack support, SCP considerations
- Can be purchased at different scopes:
 - Resource Group
 - Subscription
 - Enterprise Agreement (best practice)
 - Pool consumption across subscriptions for deeper discounts
 - Use chargeback model to charge each subscription for their exact usage



Cost Report for P3



Chargebacks

- Create separate workspaces for different divisions/teams
 - Associate workspace tag with each workspace
 - Leverage Azure Policies to ensure consistent application of workspace tags
- Create separate clusters for each project/initiative within a division/team
 - Associate cluster tags with each project
 - Leverage Databricks Cluster Policies to ensure consistent application of cluster tags
- Monitor & report chargebacks by workspace/cluster tags



Budget & Cost Monitoring

250_per_month

Subscription

Edit budget

Delete budget

CURRENT SPEND

29.81 USD

Budget

100.00 USD

Budget evaluations now include reserved instance and purchase charges. To learn more, visit [the budgets documentation](#).

BUDGET SUMMARY

Name

250_per_month

Scope

18ab4010-b0a9-4be8-95ed-da870501f427 (Subscription)

Filters

-

Amount

100.00 USD

Budget period

Resets monthly

Start date

3/1/2020

End date

2/28/2022

BUDGET ALERTS

Alert conditions

% Of budget	Amount	Action group	Action group type
50%	\$50	ajay-test-budget-action-group	1 Email
75%	\$75	ajay-test-budget-action-group	1 Email
100%	\$100	ajay-test-budget-action-group	1 Email

Define budgets by workspace & cluster tags in Azure Cost Center

Create action groups for budget thresholds

The background is a dark blue-grey color. It features several abstract geometric shapes in a vibrant orange-red and a muted teal. In the top right, there is a large orange circle. Below it, towards the center, is a small orange square. On the right side, there are several overlapping triangles and polygons in both orange and teal, creating a dynamic, layered effect. A small orange circle is located near the bottom right.

Gracias Thank you

Appendix

Unblocking your Data + AI Future

Unity Catalog is the Key



None of this
is available
without
Unity
Catalog



Managed and External Tables

Comparison

Characteristic	Managed Tables	External Tables
Syntax	Simpler syntax, no overhead, user doesn't need to know any locations or URLs: - CREATE TABLE <name> - df.write.saveAsTable("<name>")	The table location must be known and specified (full 'abfss://' URL as UC does not support mounts): - CREATE TABLE <name> LOCATION 'abfss://...' - df.write.option("path", "abfss://...").saveAsTable("<name>")
Data Location	Data lives in the metastore / catalog / schema managed storage location: - Uses GUID style folders which are less human-readable: abfss://data@storageaccount.dfs.core.windows.net/myschema/.../8d408932-2926-4c5c-9403-0fa7dd2e485d - There are multiple ways to retrieve the table location: - DESCRIBE DETAIL <table_name> - UC REST API: /api/2.1/unity-catalog/tables/{full_name} - Directly via the HMS compatible layer - Every time a managed table is dropped and recreated, it will use a different GUID folder - If using CREATE OR REPLACE instead of DROP/CREATE then the location is preserved	Data lives in a user-provided storage location - In UC, this is an External Location and users must also have the correct privileges (CREATE EXTERNAL TABLE) to be able to create a table on an External Location - Specified with LOCATION or .option("path") - However, it must be a full 'abfss://' URL as UC does not support mounts
DROP behaviour	DROP discards metadata and underlying data is deleted from the storage account within 30 days - UNDROP can be used to restore the table - If recreating the table then it's better to use CREATE OR REPLACE instead of DROP/CREATE	DROP discards metadata only - UNDROP can also be used - If data needs to be deleted then this must be managed separately (e.g. dbutils.fs.rm)
Supported formats	DELTA only	DELTA, PARQUET, ORC ,CSV, JSON, AVRO, TEXT



Managed and External Tables

Comparison cont.

Characteristic	Managed Tables	External Tables
Predictive Optimization	Coming to Managed tables in a first phase	Coming to External tables in a second phase (typically 1-2 quarters after Managed tables)
New product features	Supported first - For example - Liquid tables (partition-less auto clustering tables) - DLT only supports Managed tables	Might be supported later, no guarantees - Predictive optimization and automatic statistics aren't available on external tables - DLT doesn't support External tables
External readers	External readers could read from UC Managed tables, however the folder path is less human-readable - Recommended to use the HMS compatibility layer - If not possible, there are multiple ways to retrieve the table location: - DESCRIBE DETAIL <table_name> - UC REST API: /api/2.1/unity-catalog/tables/{full_name}	External tables are most recommended for when an external reader needs to access the data using the location and doesn't support an HMS. However, this will bypass the governance and access control enforced by UC (permissions, lineage, audit, etc.)
Migration from Hive	Currently, migrating to a Managed UC table can only happen with data copy (DEEP CLONE or CTAS) New functionality to permit Managed to Managed migration is being developed	Simpler, either with the Data Explorer UI or with the SYNC command. No data migration, only the metadata is copied
DR	Currently more complex than External tables, simpler solution in development (2025)	Simple as it can reference the same path (if data replication is handled separately)
Best for	User created Delta tables (Gold/Platinum)	- Pipelines created Delta tables (Bronze, Silver) - Tables over folders shared with other systems (not supported by UC HIVE API). But it's a security issue. - Non-Delta tables

